"Simply put, this book is both a 'must read' and a great reference for anyone working to define and manage software development projects. In today's modern software development world, too often sound requirements practices are set aside for the lure of 'unencumbered' agile. Karl and Joy have detailed a progressive approach to managing requirements, and detailed how to accommodate the ever-changing approaches to delivering software."

—Mary Kalal, Software Development Director, Garland, a Mitek Industries company

"I am so pleased to see the updated book on software requirements from Karl Wiegers and Joy Beatty. I especially like the latest topic on how to apply effective requirements practices to agile projects, because it is a service that our consultants are engaged in more and more these days. The practical guide and real examples of the many different requirement practices are invaluable."

—Doreen Lenz, Managing Director of the Requirements and Business Analysis Practice for Builders Group Inc.

"As an early adopter of Karl's classic book, *Software Requirements*, I have been eagerly awaiting his new edition—and it doesn't disappoint. Over the years, IT development has undergone a change of focus from large new 'green-field' projects towards adoption of ready-made off-the-shelf solutions and quick-release agile practices. In this latest edition, Karl and Joy explore the implications of these new developments on the requirements process with invaluable recommendations based not on dogma but on what works, honed from their broad and deep experience in the field."

—Howard Podeswa, CEO, Nobleprog, and author of The Business Analyst's Handbook

"If you are looking for a practical guide into what software requirements are, how to craft them, and what to do with them, then look no further than *Software Requirements, Third Edition*. This usable and readable text walks you through exactly how to approach common requirements related scenarios. The incorporation of multiple stories, case studies, anecdotes, and examples keeps it engaging to read."

—Jason Beaubouef, CBAP, Host of Bridging the Gap

"How do you make a good requirements read better? You add content like Karl and Joy did to address incorporating product vision, tackling agility issues, covering requirements reuse, tackling packaged and outsourced software, and addressing specific user classes. You could take an outside look inside of requirements to address process and risk issues and go beyond just capturing functionality."

—Donald J. Reifer, President, Reifer Consultants LLC

"This new edition keeps pace with the speed of business both in deepening the foundation of the second edition and in bringing analysts down to earth how-to's for addressing the surge in agile development, using features to control scope, improving elicitation techniques, and expanding modeling. Wiegers and Beatty have put together a 'must-read' for anyone in the profession."

—Kent Eis, President and CEO, Eriksen Solutions Inc. and author of Business Analyst Benchmark

Microsoft

Software Requirements, Third Edition

Karl Wiegers and Joy Beatty



Next steps

- Relate Figure 5-3 to the way you hear the voice of the user in your own environment. Do you encounter any problems with your current communication links? Identify the shortest and most effective communication paths that you can use to elicit user requirements in the future.
- Identify the different user classes for your project. Which ones are favored? Which, if any, are disfavored? Who would make a good product champion for each important user class? Even if the project is already underway, the team likely would benefit from having product champions involved.
- Starting with Table 6-2, define the activities you would like your product champions to perform. Negotiate the specific contributions with each candidate product champion and his or her manager.
- Determine who the decision makers are for requirements issues on your project. How well does your current decision-making approach work? Where does it break down? Are the right people making decisions? If not, what should be going on? Suggest processes that the decision makers should use for reaching agreement on requirements issues.

Requirements elicitation

"Good morning, Maria. I'm Phil, the business analyst for the new employee information system we're going to build for you. Thanks for agreeing to be the product champion for this project. Your input will help us a lot. So, can you tell me what you want?"

"Hmmm, what do I want?" mused Maria. "I really know where to start. The new system should be a lot faster than the old one. And you know how the old system crashes if an employee has a really long name, and we have to call the help desk and ask them to enter the name for us? The new system should take long names without crashing. Also, a new law says we can't use Social Security numbers for employee IDs anymore, so we'll have to change all of the IDs when the new system goes in. Oh, yes, it'd be great if I could get a report of how many hours of training each employee has had so far this year."

Phil dutifully wrote down everything Maria said, but his head was spinning. Maria's desires were so scattered that he wasn't sure he was getting all the requirements. He had no idea if Maria's needs aligned with the project's business objectives. And he didn't know exactly what to do with all these bits of information. Phil wasn't sure what to ask next.

The heart of requirements development is elicitation, the process of identifying the needs and constraints of the various stakeholders for a software system. Elicitation is not the same as "gathering requirements." Nor is it a simple matter of transcribing exactly what users say. Elicitation is a collaborative and analytical process that includes activities to collect, discover, extract, and define requirements. Elicitation is used to discover business, user, functional, and nonfunctional requirements, along with other types of information. Requirements elicitation is perhaps the most challenging, critical, error-prone, and communication-intensive aspect of software development.

Engaging users in the elicitation process is a way to gain support and buy-in for the project. If you're the business analyst, try to understand the thought processes behind the requirements the users state. Walk through the processes that users follow to make decisions about their work and extract the underlying logic. Make sure that everyone understands why the system must perform certain functions. Look for proposed requirements that reflect obsolete or ineffective business processes or rules that should not be incorporated into a new system.

The BA must create an environment conducive to a thorough exploration of the product being specified. To facilitate clear communication, use the vocabulary of the business domain instead of forcing customers to understand technical jargon. Record significant application domain terms in a glossary, rather than assuming that all participants share the same definitions. Customers must understand that a discussion about possible functionality is not a commitment to include it in the

product. Brainstorming and imagining the possibilities is a separate matter from analyzing priorities, feasibility, and the constraining realities. It's never too early for stakeholders to prioritize their blue-sky wish lists to avoid defining an enormous project that never delivers anything useful.

The output of requirements development is a common understanding of the needs held by the diverse project stakeholders. When the developers understand those needs, they can explore alternative solutions to address them. Elicitation participants should resist the temptation to design the system until they understand the problem. Otherwise they can expect to do considerable design rework as the requirements become better defined. Emphasizing user tasks rather than user interfaces and focusing on true needs more than on expressed desires, help keep the team from being sidetracked by prematurely specifying design details.

As Figure 7-1 shows, the nature of requirements development is cyclic. You will do some elicitation, study what you learned, write some requirements, perhaps determine that you're missing some information, perform additional elicitation, and so forth. Don't expect to just make a couple of elicitation workshops and then declare victory and move on.



FIGURE 7-1 The cyclic nature of requirements elicitation, analysis, and specification.

This chapter describes a variety of effective elicitation techniques, including when to use each one, as well as tips and challenges for each. The rest of the chapter describes the overall elicitation process, from planning elicitation activities to organizing the session outputs. Later in the chapter, we offer cautions about a few traps to watch out for during elicitation, and specific suggestions for identifying missing requirements. Figure 7-2 depicts the activities for a single requirements elicitation session. Below we walk through this process, though we'll explore some of the requirements elicitation techniques you might find valuable.

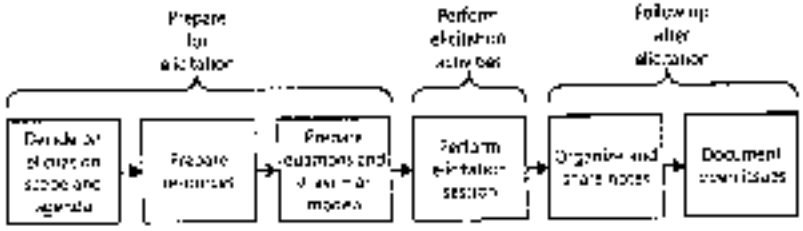


FIGURE 7-2 Activities for a single requirements elicitation session.

Requirements elicitation techniques

Numerous elicitation techniques can be employed on software projects. In fact, no project team should expect to use only one elicitation technique. There are always many types of information to be discovered, and different stakeholders will prefer different approaches. One user might be able to clearly articulate how he uses the system, whereas you might need to observe another performing her job to reach the same level of understanding.

Elicitation techniques include both facilitated activities, in which you interact with stakeholders to elicit requirements, and independent activities, in which you work on your own to discover information. Facilitated activities primarily focus on discovering business and user requirements. Working directly with users is necessary because user requirements encompass the tasks that users need to accomplish with the system. To elicit business requirements, you will need to work with people such as the project sponsor. The independent elicitation techniques supplement requirements that users present and reveal needed functionality that end users might not be aware of. Most projects will use a combination of both facilitated and independent elicitation activities. Each technique offers a different exploration of the requirements or might even reveal completely different requirements. The following sections describe several techniques commonly used to elicit requirements.

Interviews

The most obvious way to find out what the users of a software system need is to ask them. Interviews are a traditional source of requirements input for both commercial products and information systems across all software development approaches. Most BAs will facilitate some form of individual or small-group interviews to elicit requirements on their projects. Agile projects make extensive use of interviews as a mechanism to get direct user involvement. Interviews are easier to schedule and lead than large-group activities such as requirements workshops.

If you are new to an application domain, interviews with experts can help you get up to speed quickly. This will allow you to prepare draft requirements and models to use in other interviews or in workshops. If you can establish rapport with the interviewees, they will feel safer when sharing their thoughts one-on-one or in a small group than in a larger workshop, particularly about touchy topics. It's also easier to get user buy-in about participating in the project or reviewing early requirements during a one-on-one or small-group interview than in a large group setting. Interviews are appropriate for eliciting business requirements from executives who do not have a lot of time to meet.

For guidance on how to conduct user interviews, see Ian Alexander and Lerka Beus-Dukic (2009) and Howard Podeswa (2009). A few suggestions for conducting interviews follow. These are useful tips for conducting elicitation workshops as well.

Establish rapport To begin an interview, introduce yourself if the attendees don't already know you, review the agenda, remind attendees of the session objectives, and address any preliminary questions or concerns attendees have.

Stay in scope As with any elicitation session, keep the discussion focused on its objective. Even when you are talking with just one person or a small group, there's a chance the interview will go off topic.

Prepare questions and straw man models ahead of time Prepare for interviews by drafting any materials you can beforehand, such as a list of questions to guide the conversation. Draft materials will give your users a starting point to think from. People can often critique content more easily than they can create it. Preparing questions and drafting straw man models are described further in the "Preparing for elicitation" section later in this chapter.

Suggest ideas Rather than simply transcribing what customers say, a creative BA processes ideas and alternatives during elicitation. Sometimes users don't realize the capabilities developers can provide; they might get excited when you suggest functionality that will make the system especially valuable. When users truly can't express what they need, perhaps you can watch them work and suggest ways to automate portions of the job (see the "Observations" section later in this chapter). BAs can think outside the mental box that limits people who are too close to the problem domain.

Listen actively Practice the techniques of active listening (leaning forward, showing patience, giving verbal feedback, and inquiring when something is unclear) and periodically restating the main idea of a speaker's message to show your understanding of that message).

Workshops

Workshops encourage stakeholder collaboration in defining requirements. Ellen Gottesdiener (2002) defines a requirements workshop as "a structured meeting in which a carefully selected group of stakeholders and content experts work together to define, create, refine, and reach closure on deliverables (such as models and documents) that represent user requirements." Workshops are facilitated sessions with multiple stakeholders and formal roles such as a facilitator and a scribe. Workshops often include several types of stakeholders, from users to developers to testers. They are used to elicit requirements from multiple stakeholders concurrently. Working in a group is more effective for resolving disagreements than is talking to people individually. Also, workshops are helpful when quick elicitation turnaround is needed because of schedule constraints.

According to one author, "Facilitation is the art of helping people through processes toward agreed-upon objectives in a manner that encourages participation, ownership, and productivity from all involved" (Kilbert 1994). The facilitator plays a critical role in planning the workshop, selecting participants, and guiding them to a successful outcome. Business analysts frequently facilitate elicitation workshops. When a team is getting started with new approaches to requirements elicitation, consider having an outside facilitator or a second BA facilitate the initial workshops. This way the lead BA can devote his full attention to the discussion. If the sole BA is also acting as facilitator, she needs to be mindful of when she is speaking as a facilitator and when she's participating in the discussion. A scribe assists the facilitator by capturing the points that come up during the discussion. It's extremely challenging to facilitate, scribe, and participate simultaneously and do a good job on all three.

Workshops can be resource intensive, sometimes requiring numerous participants for several days at a time. They must be well planned to avoid wasting time. Minimize wasted time by coming into a workshop with drafts of materials prepared ahead of time. For example, you might draft use cases that can be reviewed as a group rather than having the entire group craft them together. Rarely does it make sense to start a workshop with a completely blank slate. Use other elicitation techniques prior to the workshops, and then bring the stakeholders together to work through only the necessary areas.

General facilitation practices apply to requirements elicitation (Schwarz 2002). A definitive resource specific to facilitating requirements elicitation workshops is Gottesdiener's *Requirements by Collaboration* (2002). She describes a wealth of techniques and tools for workshop facilitation. Following are a few tips for conducting effective elicitation workshops, many of which also apply to interviews.

Establish and enforce ground rules The workshop participants should agree on some basic operating principles. Examples include starting and ending on time, returning from breaks promptly, silencing electronic devices, holding one conversation at a time, expecting everyone to contribute, and focusing comments and criticisms on issues rather than individuals. After the rules are set, ensure that participants follow them.

Fill all of the team roles A facilitator must make sure that the following tasks are covered by people in the workshop: note taking, time keeping, scope management, ground rule management, and making sure everyone is heard. A scribe might record what's going on while someone else watches the clock.

Plan an agenda Each workshop needs a clear plan, as discussed in the "Preparing for elicitation" section later in this chapter. Create the plan and workshop agenda ahead of time and communicate them to participants so they know the objectives and what to expect and can prepare accordingly.

Stay in scope Refer to the business requirements to confirm whether proposed user requirements lie within the current project scope. Keep each workshop focused on the right level of abstraction for that session's objectives. Groups easily dive into distracting detail during requirements discussions. These discussions consume time that the group should spend on developing a higher-level understanding of user requirements; the details will come later. The facilitator will have to reel in the elicitation participants periodically to keep them on topic.

> **Trap** Watch out for off-topic discussions, such as design explorations, during elicitation sessions. Keep the participants focused on the session's objectives, while assuring them that they'll have future opportunities to work through other issues that arise.

Use parking lots to capture items for later consideration An array of relevant but important information will surface during elicitation discussions: quality attributes, business rules, user interface ideas, and more. Organize this information on flipcharts—parking lots—so you don't lose it and to demonstrate respect for the participant who brought it up. Don't be distracted into discussing off-topic details unless they turn out to be showstoppers. Describe what will happen with the parking lot issues following the meeting.

Timebox discussions Consider allocating a fixed period of time to each discussion topic. The discussion might need to be completed later but timeboxing helps avoid the trap of spending far more time than intended on the first topic and neglecting other important topics entirely. When closing a timeboxed discussion, summarize status and next steps before leaving the topic.

Keep the team small but include the right stakeholders Small groups can work much faster than larger teams. Elicitation workshops with more than five or six active participants can become stressful or argumentative. Consider running multiple workshops in parallel to explore the requirements of different user classes. Workshop participants could include the product champion and other user representatives, perhaps a subject matter expert, a BA, a developer, and a tester. Knowledge, experience, and the authority to make decisions are qualifications for participating in elicitation workshops.



Too many cooks

Requirements elicitation workshops that involve too many participants can slow into a contentious crawl. My colleague Debbie was frustrated at the sluggish progress of the first use case workshop she had taken for a website project. The 12 participants held extended discussions of unnecessary details and couldn't agree on how each use case ought to work. The team's progress accelerated nicely when Debbie reduced the number of participants to about six who represented the key roles of analyst, customer, system architect, developer, and visual designer. The workshop lost some input by using the smaller team, but the rate of progress more than compensated for that loss. The workshop participants should exchange information offline with colleagues who don't attend and then bring the collective input to the workshops.

Keep everyone engaged Sometimes certain participants will stop contributing to the discussion. These people might be frustrated for a variety of reasons. Perhaps their input isn't being taken seriously because other participants don't find their concerns interesting, or maybe they don't want to disrupt the work that the group has completed so far. Perhaps the stakeholder who has withdrawn is deferring to more aggressive participants or a domineering analyst. The facilitator must read the body language (lack of eye contact, fidgeting, yawning, checking the clock) to understand why someone has turned out of the process, and try to re-engage the person. Visual cues are absent when you are facilitating via a teleconference, so you have to listen carefully to learn who is not participating and the tones being used. You must ask those silent individuals directly if they have any thoughts about the discussion they'd like to share. The facilitator must ensure that everyone is heard.

Focus groups

A focus group is a representative group of users who converse in a facilitated elicitation activity to generate input and views on a product's functional and quality requirements. Focus group sessions must be interactive, allowing all users a chance to voice their thoughts. Focus groups are useful for exploring users' attitudes, impressions, preferences, and needs (IIBA 2009). They are particularly valuable if you are developing commercial products and don't have ready access to end users within your company.

When conflicts erupt

Differing perspectives, priorities, and personalities can lead to conflict and even anger within a group. If this happens, deal with it immediately. Look for nonverbal cues showing conflict or anger and try to understand the cause. When the group is clear on the reason for the conflict, you might be able to find a solution to it (if one is needed).

If an individual simply will not participate in a productive way, talk with him privately to determine whether his presence will prevent the group from moving forward. If so, you might need to thank the person for his time and continue without him. Sometimes this will not be an option, and you need to simply abandon the session or topic completely for now. Conflict management is a complex skill to develop, and there are numerous resources on this (Fisher, Ury, and Patton 2011, Patterson et al. 2012).



I once scheduled a session to elicit business requirements from a new director of sales. He was known to have an antagonistic personality, so I came to the meeting prepared to really listen to and understand his desires. In the very first minute of the meeting, he started yelling at me, asking why we were holding this meeting at all. He said, "Who are you to think you have a right to ask me about my business objectives?" I took a deep breath and a long pause. Then I tried to explain why I needed to understand his business objectives—that without them, the team would be guessing at what we needed to develop to meet the customers' desires, and he would be sorely disappointed with the results. And as fast as he got mad, he got over it. Without hesitation, he started rattling off his business objectives. Thankfully my scribe was there to catch them because I was still a bit taken aback by the whole exchange.

Often, you will have a large and diverse user base to draw from, so select the focus group members carefully. Include users who have used previous versions or products similar to the one you're implementing. Either select a pool of users who are all of the same type (and hold multiple focus groups for the different user classes) or select a pool representing the full spectrum of user classes so everyone is equally represented.

Focus groups must be facilitated. You will need to keep the discussion on topic, but without influencing the opinions being expressed. You might want to record the session so you can go back and listen carefully to comments. Do not expect quantitative analysis from focus groups, but rather a list of subjective feedback that can be further evaluated and prioritized as requirements are developed. Elicitation sessions with focus groups benefit from many of the same tips described previously for workshops. Participants in focus groups normally do not have decision-making authority for requirements.

Observations

When you ask users to describe how they do their jobs, they will likely have a hard time being precise—details might be missing or incorrect. Often this is because tasks are complex and it's hard to remember every minute detail. In other cases, it is because users are so familiar with executing a

task that they can't articulate everything they do. Perhaps the task is so habitual that they don't even think about it. Sometimes you can learn a lot by observing exactly how users perform their tasks.

Observations are time consuming, so they aren't suitable for every user or every task. To avoid disrupting the users' regularly assigned work activities, limit each observation time to two hours or less. Select important or high-risk tasks and multiple user classes for observations. If you use observations in agile projects, have the user demonstrate only the specific tasks related to the forthcoming iteration.

Observing a user's workflow in the task environment allows the BA to validate information collected from other sources, to identify new issues for interviews, to see problems with the current system, and to identify ways that the new system can better support the workflow. The BA must abstract and generalize beyond the observed user's activities to ensure that the requirements captured apply to the user class as a whole, not just to that individual's. A skilled BA can also often suggest ideas for improving the user's current business processes.

Watch me bake a cake

To demonstrate the power of observations, tell some friends the steps to bake a cake from a mix. You'll likely remember the steps to turn on the oven, get out the necessary dishes and utensils, add each ingredient, mix the ingredients, prepare the pan, put the batter in the pan, bake it, and pull it out of the oven when done. But when you told your friends to add each ingredient, did you remember to say to open the bag with the mix in it? Did you remember to say to crack the eggshell, add only the contents of the egg, and discard the shell? These seemingly obvious steps might not be so obvious to someone who has never baked before.

Observations can be silent or interactive. Silent observations are appropriate when busy users cannot be interrupted. Interactive observations allow the BA to interrupt the user, ask a question. This is useful to understand immediately why a user made a choice or to ask him what he was thinking about when he took some action. Document what you observe for further analysis after the session. You might also consider video recording the session, if policies allow, so you can refresh your memory later.

I was developing a call-center application for customer service representatives (CSRs) who were used to having to page through printed catalogs to find products that customers wanted to order. The BA team met with several CSRs to elicit use cases for the new application. Each one told how difficult it was to have to flip through multiple catalogs to find exactly what product a customer was referring to. Each BA sat with a different CSR while the CSRs took orders over the phone. We saw the difficulty they faced by watching them first try to find the catalog by date, then try to locate the right product. The observation sessions helped us understand what features they would need in an online product catalog.

Questionnaires

Questionnaires are a way to survey large groups of users to understand their needs. They are inexpensive, making them a logical choice for eliciting information from large user populations, and they can be administered easily across geographical boundaries. The analyzed results of questionnaires can be used as an input to other elicitation techniques. For example, you might use a questionnaire to identify users' biggest pain points with an existing system, then use the results to discuss prioritization with decision makers in the workshop. You can also use questionnaires to survey commercial product users for feedback.

Preparing well-written questions is the biggest challenge with questionnaires. Many tools are available for writing questionnaires (Colorado State University 2013), and we suggest the most important ones here:

- Provide answer options that cover the full set of possible responses.
- Make answer choices both mutually exclusive (no overlaps in numeric ranges) and exhaustive (list all possible choices and/or have a write-in spot for a choice you didn't think of).
- Don't phrase a question in a way that implies a "correct" answer.
- If you use scales, use them consistently throughout the questionnaire.

  Use closed questions with two or more specific choices if you want to use the questionnaire results for statistical analysis. Open-ended questions allow users to respond any way they want, so it's hard to look for commonalities in the results.
- Consider consulting with an expert in questionnaire design and administration to ensure that you ask the right questions of the right people.
- Always test a questionnaire before distributing it. It's frustrating to discover too late that a question was phrased ambiguously or to realize that an important question was omitted.
- Don't ask too many questions or people won't respond.

Grupo 44

System interface analysis

Interface analysis is an independent elicitation technique that entails examining the systems to which your system connects. System interface analysis reveals functional requirements regarding the exchange of data and services between systems (IIBA 2013b). Context diagrams and ecosystem maps (see Chapter 5, "Establishing the Business Requirements") are an obvious choice to begin finding interfaces for further study. In fact, if you find an interface that has associated requirements and that is not represented in one of these diagrams, the diagrams are incomplete.

For each system that interfaces with yours, identify functionality in the other system that might lead to requirements for your system. These requirements could describe what data to pass to the other system, what data is received from it, and rules about that data, such as validation criteria. You might also discover existing functionality that you do not need to implement in your system. Suppose you thought you needed to implement validation rules for a shopping cart order in an e-commerce

website before passing it to an order-management system. Through system interface analysis you might learn that multiple systems pass orders to the order-management system, which performs the validation, so you don't need to build this function.

User interface analysis

User interface (UI) analysis is an independent elicitation technique in which you study existing systems to discover user and functional requirements. It's best to interact with the existing systems directly, but if necessary you can use screen shots. User manuals for purchased packaged-software implementations often contain screen shots that will work fine as a starting point. If there is no existing system, you might be able to look at user interfaces of similar products.

When working with packaged solutions or an existing system, UI analysis can help you identify a complete list of screens to help you discover potential features. By navigating the existing UI, you can learn about the common steps users take in the system and draft use cases to review with users. UI analysis can reveal pieces of data that users need to see. It's a great way to get up to speed on how an existing system works (unless you need a lot of training to do so). Instead of asking users how they interact with the system and what steps they take, perhaps you can reach an initial understanding yourself.

Do not assume that certain functionality is needed in the new system just because you found it in an existing one. Furthermore, do not assume that because the UI forces or flows a certain way in the current system that it must be implemented that way in the future system.

Grupo 45

Document analysis

Document analysis entails examining any existing documentation for potential software requirements. The most useful documentation includes requirements specifications, business processes, lessons learned collections, and user manuals for existing or similar applications. Documents can describe corporate or industry standards that must be followed or regulations with which the product must comply. When replacing an existing system, past documentation can reveal functionality that might need to be retained, as well as obsolete functionality. For packaged-solution implementations, the vendor documentation mentions functionality that your users might need, but you might have to further explore just how to implement it in the target environment. Comparative reviews point out shortcomings in other products that you could address to gain a competitive advantage. Problem reports and enhancement requests collected from users by help desk and field support personnel can offer ideas for improving the system in future releases.

Document analysis is a way to get up to speed on an existing system or a new domain. Doing some research and drafting some requirements beforehand reduces the elicitation meeting time needed. Document analysis can reveal information people don't tell you, either because they don't think of it or because they aren't aware of it. For example, if you are building a new call center application, you might find some complicated business logic described in the user manual for an existing application. Perhaps users don't even know about this logic. You can use the results of this analysis as input to user interviews.

A risk with this technique is that the available documents might not be up to date. Requirements might have changed without the specifications being updated, or functionality might be documented that is not needed in a new system.

Fim Grupo 45

Planning elicitation on your project

Early in a project, the business analyst should plan the project's approach to requirements elicitation. Even a simple plan of action increases the chance of success and sets realistic expectations for the stakeholders. Only by gaining explicit commitment can elicitation resources, schedule, and deliverables be set, you avoid having participants called away to do other work. An elicitation plan includes the techniques you'll use, when you plan to use them, and for what purpose. As with any plan, use it as a guide and reminder throughout the project, but realize that you might need to change the plan throughout the project. Your plan should address the following items.

- **Elicitation objectives.** Plan the elicitation objectives for the entire project and the objectives for each planned elicitation activity.
- **Elicitation strategy and planned techniques.** Decide which techniques to use with different stakeholder groups. You might use some combination of questionnaires, workshops, customer visits, individual interviews, and other techniques, depending on the access you have to stakeholders, time constraints, and your knowledge of the existing system.
- **Schedule and resource estimates.** Identify both customer and development participants for the various elicitation activities, along with estimates of the effort and calendar time required. You might only be able to identify the user classes and not specific individuals up front, but that will allow managers to begin planning for upcoming resource needs. Estimate the BA time, including time to prepare for elicitation and to perform follow-up analysis.
- **Documents and systems needed for independent elicitation.** If you are conducting document, system interface, or user interface analysis, identify the materials needed to ensure that you have them when you need them.
- **Expected products of elicitation efforts.** Knowing you are going to create a list of use cases, an SRS, an analysis of questionnaire results, or quality attribute specifications helps ensure that you target the right stakeholders, topics, and details during elicitation.
- **Elicitation risks.** Identify factors that could impede your ability to complete the elicitation activities as intended, estimate the severity of each risk, and decide how you can mitigate or control it. See Chapter 23, "Software requirements and risk management," for more on risk management. See Appendix B, "Requirements troubleshooting guide," for symptoms, root causes, and possible solutions for common elicitation problems.

Many BAs have their go-to elicitation techniques—commonly interviews and workshops—and do not think to use other techniques that might reduce resource needs or increase the quality of the information discovered. Rarely will a BA get the best results by using only one elicitation technique or

a project. Elicitation techniques apply across the spectrum of development approaches. The selection of elicitation techniques should be based on the characteristics of the project.

Figure 7-3 suggests the elicitation techniques that are most likely to be useful for various types of projects. Select the rows or rows that represent characteristics of your project and read to the right to see which elicitation techniques are most likely to be helpful (marked with an X). For instance, if you're developing a new application, you're likely to get the best results with a combination of stakeholder interviews, workshops, and system interface analysis. Most projects can make use of interviews and workshops. Focus groups are more appropriate than workshops for mass market software (because you have a large external user base but limited access to representatives). These suggestions for elicitation techniques are just that—suggestions. For instance, you might conclude that you don't want to apply user interface analysis on mass market software projects.

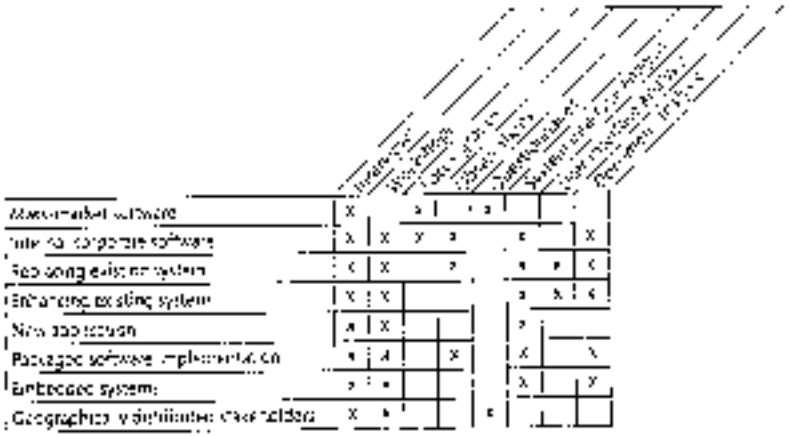


FIGURE 7-3 Suggested elicitation techniques by project characteristic.

Grupo 46

Preparing for elicitation

Facilitated elicitation sessions require preparation to make the best use of everyone's time. The larger the group participating in the session, the more important preparation is. Figure 7-4 highlights the activities to prepare for a single requirements elicitation session.

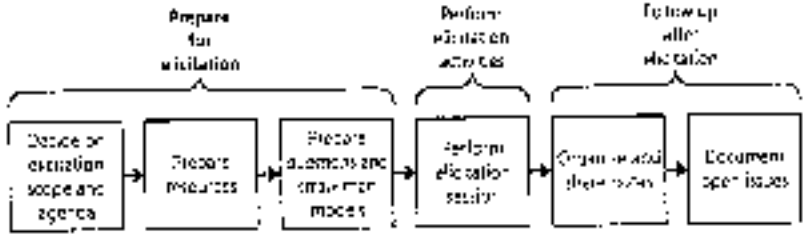


FIGURE 7-4 Activities to prepare for a single elicitation session.

Prepare for each session by deciding on the scope of the session, communicating an agenda, preparing questions, and drafting materials that might be used during the session. The following tips will help you prepare for elicitation.

Plan session scope and agenda Decide on the scope of the elicitation session, taking into account how much time is available. You might define the session scope by using a set of topics or questions or you might list a specific set of process flows or use cases to be explored. Align the scope of the session with the overall project scope defined in the business requirements so you can keep the conversation on topic. The agenda should itemize what topics will be covered, the available time for each topic, and targeted objectives. Share the session agenda with stakeholders in advance.

Prepare resources Schedule the physical resources needed, such as rooms, projectors, teleconference numbers, and videoconferencing equipment. Also, schedule the participants, being sensitive to time zone differences if you are not all in the same location. For geographically dispersed groups, change the schedule each time you meet so the sessions do not always inconvenience the same people in a particular part of the world. Collect documentation from various sources. Gain access to systems as necessary. Take online training to learn about existing systems.

Grupo 47

Learn about the stakeholders Identify the relevant stakeholders for the session (see Chapter 6, "Finding the voice of the user"). Learn about the stakeholders' cultural and regional preferences for meetings. If some of the participants are not native speakers of the language in which the session will be conducted, consider providing them with supporting documentation, such as slides, ahead of time so they can read ahead or follow along. The slides can list specific questions you will be asking or simply provide context for the session that you might also verbally explain. Avoid creating an "us" versus "them" tension.

Prepare questions Go into every facilitated elicitation session with a set of prepared questions. Use areas of uncertainty in straw man models (described in the next section) as a source of questions. If you are preparing for an interview or workshop, use results from other elicitation techniques to identify unresolved questions. There are many sources of suggested questions for elicitation (Wiegers 2006; Miller 2009).

Phrase your questions to avoid leading customers down an unintended path or toward a specific answer. As an analyst, you must probe beneath the surface of the requirements the customers present to understand their true needs. Simply asking users, "What do you want?" generates a mass of random information that leaves the analyst floundering. "What do you need to do?" is a much better question. Asking "why" several times can move the discussion from a presented solution to a solid understanding of the problem that needs to be solved. Ask open-ended questions to help you understand the user's current business processes and to see how the new system could improve their performance.

Imagine yourself learning the user's job, or actually do the job under the user's direction. What tasks would you perform? What questions would you have? Another approach is to play the role of an apprentice, learning from a master user. The user you are interviewing then guides the discussion and describes what he views as the important topics for discussion.

Probe around the exceptions. What could prevent the user from successfully completing a task? How should the system respond to various error conditions? Ask questions that begin with "What else could . . ." "What happens when . . ." "Would you ever need to . . ." "Where do you get . . ." "Why

Grupo 48

do you (or don't you), . . . ," and "Does anyone ever . . ." Document the source of each requirement so that you can obtain further clarification if needed and trace development activities back to specific customer origins.

As with any improvement activity, dissatisfaction with the current situation provides a springboard for the new and improved future state. When you're working on a replacement project for a legacy system, ask the users, "What three things annoy you the most about the existing system?" This question surfaces expectations that the users hold for the follow-on system.

You won't have—nor do you need—a perfect script going into an interview or a workshop. The three sets of questions are to help you if you get stuck. The questions should seem natural and comfortable—like a conversation, not an interrogation. Five minutes into a session, you might realize that you missed an important area for discussion. Be ready to abandon your questions if needed. At the end of your session, ask "Is there anything else you expected me to ask?" to try to surface issues you just didn't think of.

Prepare straw man models Analysis models can be used during elicitation sessions to help users provide better requirements. Some of the most useful models are use cases and process flows because they closely align with how people think about doing their jobs. Create straw man or draft models ahead of your elicitation sessions. A straw man serves as a starting point that helps you learn about the topic and inspires your users to think of ideas. It's easier to revise a draft model than to create one from scratch.

If you are new to the project's domain, it might be hard to create a draft model on your own. Use other elicitation techniques to glean enough knowledge to work from. Read existing documents, examine existing systems for models you can reuse as a starting point, or hold a one-on-one interview with a subject matter expert to learn enough to get started. Then tell the group you're working with, "This model will probably be wrong. Please tear it apart and tell me how it should look. You won't hurt my feelings."

Grupo 49

Performing elicitation activities

Figure 7-5 highlights the activity to perform elicitation in a single requirements elicitation session.

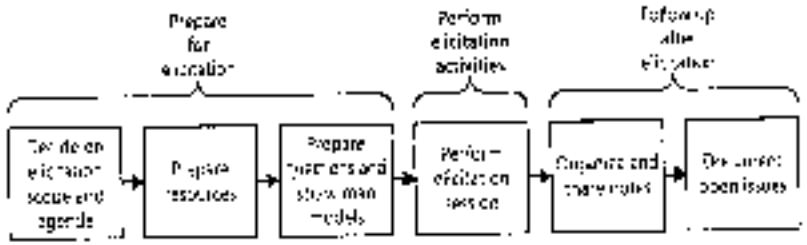


FIGURE 7-5 The perform elicitation activities step for a single elicitation session

Executing the elicitation activity itself is relatively obvious—if you are interviewing, you talk to people; if you are performing document analysis, you read the document. However, when facilitating an elicitation activity, the following tips might be useful:

Educate stakeholders Teach your stakeholders about your elicitation approach and why you chose it. Explain the elicitation techniques you'll be using, such as use cases or process flows, and how they can help stakeholders provide better requirements. Also describe how you will capture their information and send them materials for review after the session.

Take good notes Assign someone who isn't actively participating in the discussion to be the scribe responsible for taking accurate notes. Session notes should contain an attendee list, notes on who did not attend, decisions made, actions to be taken and who is responsible for each, outstanding issues, and the high points of key discussions. Unfortunately, BAs sometimes hold facilitated elicitation sessions without a dedicated scribe and have to fill the role themselves. If you're in this situation, be prepared to write shorthand, type fast, or use a recording device (if the participants agree). Audio sessions can be transcribed into notes for entry into a form and be sent to the recorded audio discussion. You can also use whiteboards and paper on the walls and photograph them.

Grupo 50

Prepare questions ahead of time to eliminate some of the on-the-spot thinking necessary to keep the conversation going. Come up with a shorthand notation to capture a question that comes to mind while someone is talking, so you can quickly flip back to it when you have an opportunity. Don't try to capture diagrams in complicated diagramming software; just photograph sketches, diagrams, or draw quickly by hand.

Exploit the physical space Most rooms have four walls, so use them during facilitation to draw diagrams or create lists. If there aren't whiteboards available, attach big sheets of paper to the walls. Have sticky notes and markers available. Invite other participants to get up and contribute to the wall as well; moving around helps to keep people engaged. Gottesdiener (2005) refers to this technique as the "Wall of Wonder" collaboration pattern. If there are existing artifacts to look at (such as straw man models, existing requirements, or existing systems), project them on the wall.

Facilitating collaborative sessions with participants in multiple locations requires more creativity. You can use online conferencing tools to share slides and permit interactions. If several participants are in the same room, use videoconferencing tools to show remote participants what's on the walls and whiteboards.

Stakeholders on the move

Once I facilitated a workshop to elicit process flows for a semiconductor fabrication plant with a dozen engineers. I started out by working at the whiteboard, drawing the flows as we talked. Each time we completed a flow, I'd stop to photograph it before moving on to the next. Half a day into the first session, one of the engineers asked if he could have a turn at the whiteboard. I happily handed him the marker. He had learned the flowchart notation, and since he was already an expert in the system, he could easily draw the flow on the board. He then walked us through it, asking his peers at each step to validate or correct it. He was leading the process, which allowed me to focus on asking probing questions and taking notes. Soon, all the engineers were passing the marker around, so everyone got a turn.